

Project Guide: Workload-Aware Low-Power BIST (WA-LP-BIST)

This document explains exactly what your project is, why it matters, and how it works in plain, easy-to-understand language. You can easily export this Markdown file to a PDF in your editor.

1. The Big Picture: What are you doing?

You have invented and implemented a **new methodology for testing microchips** after they are manufactured in a silicon foundry.

Specifically, you designed a **"Workload-Aware" Built-In Self-Test (BIST)**. It is a way for a microchip to test itself for physical manufacturing defects (like broken wires) *without* drawing massive amounts of electrical power and overheating during the test.

To prove that your new testing methodology actually works, you built a **32-bit RISC-V ALU** (the mathematical brain of a processor) and wrapped your testing framework around it as a real-world case study.

2. The Problem: Why does the industry need this?

When millions of chips are manufactured, some will have microscopic defects. To find the broken chips, they must be tested.

The Old Way (Standard BIST):

Usually, engineers put a random number generator inside the chip (called an LFSR). This blasts the chip with completely random 1s and 0s at billions of times per second.

- **The issue:** Because the inputs are totally random, the chip is forced to rapidly jump between doing completely different tasks on every single clock cycle (e.g., Add -> Shift -> XOR -> Subtract).
- **The result:** This violent switching (called "thrashing") causes almost every transistor in the chip to turn on and off simultaneously. The chip draws drastically more power during this test than it ever would when running normal software. It can literally overheat and melt, or falsely fail the test because of voltage drops.

3. The Solution: "Workload-Aware" BIST

Your project solves this overheating problem using a **Workload-Aware** approach.

The Car Analogy:

- Standard BIST is like testing a car transmission by shifting randomly from Drive to Reverse to 5th Gear as fast as you can. It tests the gears, but it destroys the transmission.
- Your **Workload-Aware BIST** tests the car by driving it on a test track that mimics normal city and highway driving.

How you did it in hardware:

Real software programs (like the famous CoreMark and Dhrystone benchmarks) don't execute random instructions. They execute a lot of additions (for loops, counters), some logic operations, and a few shifts.

Your project includes a **Workload Mapper**. This module takes the random test data and forces it to look like real software:

- 50% Arithmetic operations
- 25% Logic operations
- 12.5% Shifters
- 12.5% Comparators

By clustering these similar operations together, your test smoothly checks every part of the chip without violent electrical thrashing.

4. How Your Design Works (The Architecture)

Your hardware design is composed of several blocks working together. If you look at your code (`bist_top.sv`), here is what each piece is doing:

1. **The LFSR (Linear Feedback Shift Register):** The engine that generates the raw, random test patterns.
2. **The Workload Mapper (Your Secret Weapon):** Takes the raw random data and "calms it down" to mimic the real-world software profile mentioned above.
3. **The CUT (Circuit Under Test):** Currently, this is your `alu_rv32i` module. This is the actual hardware being tested. Because your design is modular, you could easily swap this out for a different chip later.
4. **The Fault Injector:** A module you built specifically to simulate physical manufacturing defects (like a wire being stuck at '0' or '1'). You use this to prove that your test actually catches broken hardware.
5. **The MISR (Multiple Input Signature Register):** The "checker." It compresses all the outputs of the ALU into a single 32-bit signature at the end of the test. If this signature matches the "Golden Signature," the chip is perfect. If it doesn't match, the chip is defective.
6. **The WSA Monitor:** Your power meter. It counts the number of transistor toggles (transitions). You use this to mathematically prove to the world that your Workload-Aware mode uses far less power than the Standard mode.

5. Summary of Your Achievements

If someone asks you what you accomplished, you can tell them:

"I designed a Low-Power BIST methodology that uses workload-profiling to reduce switching activity during manufacturing test. I implemented the entire BIST architecture in SystemVerilog, integrated it with a RISC-V ALU, and included on-chip hardware monitors to mathematically prove a significant reduction in power consumption without losing fault coverage."